RUPEEWISE

Google AI Studio Master Development Prompt

Personal Finance & Management Website | Java • Spring Boot • OOP • INR

Usage: Upload this PDF to Google AI Studio and instruct it to follow the complete specification. The prompt requires a real Java backend; displaying Java snippets in the UI alone does not satisfy the requirement.

1. Project Overview

Develop a complete, professional, highly interactive, beginner-friendly yet technically impressive web application named **RupeeWise — Personal Finance & Management System**.

This is an educational software engineering project that will be demonstrated to a college teacher. It must function as a real personal finance management website and simultaneously demonstrate extensive use of Java, Object-Oriented Programming (OOP), data structures, database management, software architecture, data analysis, and interactive data visualization.

Do not create merely a static UI, landing page, or dashboard prototype. Build a working application with functional forms, persistent transaction records, calculations, filters, charts, editing capabilities, and a comprehensive code documentation section.

The application must be easy to demonstrate live during a college project presentation.

2. Mandatory Technology Stack

Backend

- Java 21 or a compatible Java LTS version.
- Spring Boot for REST API development.
- Maven for dependency management and building the application.
- Java classes and objects extensively throughout the business logic.
- Spring Data JPA and Hibernate for database operations.
- H2 database for easy local setup, with persistent file-based storage enabled.
- Jakarta Bean Validation for input validation.
- JUnit 5 for unit testing.

Frontend

- HTML5, CSS3, and JavaScript.
- A clean, responsive, interactive frontend.
- Chart.js for interactive charts and graphs.
- Fetch API to communicate with the Java backend REST endpoints.

Architecture

Implement a clear three-layer architecture:

1. Presentation layer: frontend interface.

2. Business logic layer: Java services, domain objects, calculations, and validation.
3. Data access layer: repositories and persistent database storage.

Use DTOs (Data Transfer Objects) to transfer information between frontend and backend where appropriate.

IMPORTANT: Do not implement the core application in JavaScript and merely display Java code as decoration. Java must implement the real business logic, financial calculations, transaction processing, analytics, and persistence workflows.

Do not substitute Java with Python, Node.js, or another backend language.

3. Currency and Regional Settings

The entire application must operate exclusively in Indian Rupees (INR).

- Display all monetary amounts using ■.
- Format currency according to the Indian numbering convention, such as ■1,25,000.00.
- Use Indian locale en-IN.
- Use Asia/Kolkata as the application timezone.
- Use DD-MM-YYYY as the default date display format.
- Never display USD, EUR, GBP, or any other currency.
- Never perform currency conversion.
- Store monetary values using Java BigDecimal rather than double or float.
- Use appropriate precision and rounding rules for financial calculations.
- Validate that expenses, incomes, savings, and budgets cannot contain invalid negative amounts.
- Treat transfers between accounts separately from income and expenses so that transfers do not inflate spending or earnings.

4. User Profile and Initial Setup

On first launch, show a friendly setup screen. Allow the user to enter and edit:

- Full name.
- Email address, optionally.
- Monthly income target.
- Initial savings balance.
- Monthly savings goal.
- Preferred budgeting method.
- Financial year start month.
- Monthly expense budget.
- Short-term and long-term financial goals.

Include a profile page to update information. Use a default display name such as "Your Name" until the user enters details. The user should be able to reset or edit the sample profile at any time.

Do not require real bank credentials, Aadhaar numbers, PAN numbers, or other sensitive identity information.

5. Main Navigation

Create a persistent sidebar on desktop and a responsive navigation menu on smaller screens. Include:

1. Dashboard

2. Transactions
3. Ledger
4. Income
5. Expenses
6. Savings
7. Budgets
8. Financial Goals
9. Accounts
10. Reports & Analytics
11. Recurring Transactions
12. Specs
13. Settings

The selected section must be highlighted. Navigation must work correctly without broken links or placeholder pages. Use smooth but restrained transitions, clear icons, informative tooltips, and friendly empty states.

6. Dashboard

Create a comprehensive financial dashboard with a polished, modern design.

At the top, display a personalized greeting: "Good morning, [Name]! Here is your financial overview." Use the appropriate greeting based on the current local time.

Summary cards

Display:

- Total available balance.
- Total savings.
- Current month's income.
- Current month's expenses.
- Current month's net cash flow.
- Remaining monthly budget.
- Savings rate.
- Outstanding planned payments, where applicable.

Every card must show actual calculated data from the database. Provide comparison indicators against the previous month when sufficient data exists.

Interactive charts

Implement using Chart.js:

A. Expense breakdown pie or doughnut chart

Categories: Food and groceries; Education; Transportation; Housing and utilities; Healthcare; Shopping; Entertainment; Personal care; Subscriptions; Miscellaneous. Make chart segments interactive through tooltips. Selecting a category should allow the user to view corresponding transactions.

B. Monthly income versus expenses bar chart

Display the previous six months with separate bars for income and expenses. Update automatically when transactions change.

C. Savings trend line chart

Display savings balances over time. Calculate historical balances using opening balances and recorded transactions. Do not fabricate historical balances from the current balance.

D. Budget utilisation chart

Compare each category's budget against actual spending and highlight categories exceeding their budgets.

E. Income source chart

Visualise income by source, such as tuition, salary, freelance work, gifts, or other sources.

F. Cash-flow chart

Display net income after expenses for each month.

If there is insufficient data, show a useful empty-state message rather than inventing chart values.

Dashboard controls

Include date-range filters; current month, previous month, last six months, and custom range; category filters; account filters; refresh action; quick-add transaction button; export report button; recent transactions; upcoming recurring payments; budget warnings; savings goal progress; and financial insights generated from actual stored records.

All charts and summary cards must update when data is added, edited, or deleted.

7. Transaction Management

Create a fully functional transaction management system.

Each transaction should support:

- Unique transaction ID.
- Transaction date.
- Transaction type: income, expense, or transfer.
- Amount in INR.
- Category.
- Description.
- Payment method.
- Associated account.
- Optional notes and tags.
- Creation timestamp.
- Last updated timestamp.

Add transaction form

Provide a friendly modal or dedicated form. The user should be able to add income, record expenses, transfer money between accounts, select or create a category, select a payment method, choose a date, add notes, save, or cancel. Validate amount, date, transaction type, and required fields. Show understandable errors.

Edit and delete

Allow users to edit existing transactions, delete after confirmation, duplicate a transaction, search, filter by type/category/account/date range/amount, sort by date or amount, and paginate large lists.

Deleting or editing a transaction must immediately update all dependent balances, reports, charts, budgets, and savings calculations. Use database transactions where necessary to maintain financial consistency.

8. Ledger

Build a dedicated ledger section resembling a professional financial register.

Columns:

- Date.
- Transaction ID.
- Description.
- Category.
- Debit.
- Credit.
- Account.
- Running balance.
- Notes.

Accounting presentation:

- Expenses appear as debits.
- Income appears as credits.
- Transfers are represented as movements between accounts and must not be counted as income or expenses.

Allow account and date-range selection. Recalculate the running balance chronologically whenever records change.

Include opening balance, total credits, total debits, closing balance, search and filters, pagination, CSV export, and a print-friendly view. The ledger must be derived from persisted transaction records, not hardcoded rows. Ensure closing balance reconciles with the account balance.

9. Income Management

Create an income-specific page. Allow income from salary, tuition, freelancing, scholarships, business income, interest, gifts, and other sources.

Include total income for a selected period, income source breakdown, monthly trends, expected versus received income, income history, search/filtering, and add/edit/delete functionality.

Income records must be linked to the main transaction system rather than stored as disconnected duplicate records.

10. Expense Management

Create a separate expense analysis page with category-wise breakdown, daily and monthly summaries, highest spending categories, largest individual expenses, average daily expenditure, fixed versus variable expenses, essential versus discretionary expenses, expense trends, expense history, and add/edit/delete functionality.

Provide practical insights based on actual records, such as: "You spent ■850 more on dining this month than last month." Never generate an insight unless stored data supports it.

11. Savings Management

Create a dedicated savings section tracking total savings, savings by account, savings goal, monthly contributions, withdrawals, savings rate, progress towards financial goals, and historical balances.

Allow multiple savings goals. Each goal should contain goal name, target amount in INR, current amount, target date, priority, notes, progress percentage, and remaining amount.

Examples: emergency fund, laptop, higher education, travel, or personal purchase.

A contribution to a goal must not be counted as new income if the money already exists in the user's accounts. Distinguish total financial assets, account balances, and amounts earmarked for goals. Avoid double-counting the same money.

12. Budget Management

Create a monthly budgeting module. Users should set an overall monthly budget and category budgets, edit limits, compare planned spending with actual spending, view remaining budget and utilisation percentages, receive warnings at 75%, 90%, and 100%, compare budgets with previous months, and optionally carry forward unused budgets only when explicitly enabled.

Use clear progress bars and accessible warning indicators. Budget alerts must be calculated from actual expenses for the selected period.

13. Account Management

Allow users to create and manage:

- Cash wallet.
- Savings bank account.
- Current bank account.
- Other manually tracked accounts.

Each account must have account ID, account name, account type, opening balance, current balance, creation date, and active/archived status.

Transfers must work correctly. For example, transferring ■2,000 from a bank account to a cash wallet decreases the bank balance and increases the cash balance by ■2,000 without increasing total income or expenses. Prevent invalid transfers and accidental duplicate processing.

The application must not pretend to connect to a real bank. Account information is entered manually unless a genuine integration is deliberately implemented.

14. Recurring Transactions

Support daily, weekly, monthly, and yearly recurring income and expenses, such as tuition income, subscriptions, rent, and gym membership.

Provide next scheduled date, expected amount, active/paused status, edit/delete controls, and in-app upcoming payment reminders. Do not automatically record future payments as completed. Let the user confirm a due transaction or explicitly enable a clearly documented automatic posting rule.

15. Reports and Financial Analytics

Include monthly financial summary, income versus expenses, savings performance, category-wise expenses, budget performance, cash-flow analysis, account balance summary, financial goal progress, and spending trends.

Provide downloadable CSV reports and print-friendly summaries.

Calculate where possible:

- Savings rate = (Income – Expenses) / Income × 100

- Net cash flow = Income − Expenses
- Budget utilisation = Actual category expenses / Category budget × 100
- Monthly expense growth = (Current month expenses − Previous month expenses) / Previous month expenses × 100

Handle zero-income and zero-budget cases safely; never divide by zero. Explain each metric and the period used. Implement all calculations in Java and test them.

16. Intelligent Financial Insights

Create a rule-based financial insights engine in Java. Examples: a category has increased compared with the previous month; monthly budget exceeded; savings goal approaching target; income below target; recurring payments approaching; or a category consumes a large share of spending.

Insights must be based on actual data and identify the comparison period. Do not make unsupported predictions, fabricate financial advice, or claim to use AI unless an actual AI integration exists. Allow users to dismiss an insight.

17. The Specs Section — Essential Requirement

Create a dedicated navigation item named **Specs**. This is a core project feature, not an optional help page. It must let me show my teacher the actual source code, explain the implementation, and return to the dashboard to demonstrate the working application.

A. Project overview

Display project name, objectives, technology stack, architecture diagram, folder structure, data flow between frontend/backend/services/database, database entity relationships, and API endpoint documentation.

B. Complete source-code explorer

Create a searchable, categorised source-code explorer showing actual implementation files:

1. Domain models and entities.
2. Abstract classes and interfaces.
3. Inheritance and polymorphism examples.
4. Services and business logic.
5. Repositories and database access.
6. REST controllers and APIs.
7. DTOs and validation.
8. Financial analytics and calculations.
9. Exception handling.
10. Configuration and application entry point.
11. Unit tests.
12. Frontend HTML, CSS, and JavaScript.
13. Database schema and configuration.

For each file, show exact filename, relative path, language, purpose, complete actual source code, explanation of important methods, OOP concepts demonstrated, and connections to other classes.

Implement syntax highlighting, line numbers, a copy-code button, and expandable code sections.

The code displayed in Specs must be sourced from actual project files or generated from a single source of truth. Do not create a separate set of illustrative snippets that misrepresents the implementation.

If the development environment cannot expose source files dynamically at runtime, generate a synchronised source-code catalogue from actual project files during the build process. Make this catalogue part of the delivered project.

Never expose database passwords, credentials, API keys, or other secrets in the code explorer.

C. Explain every OOP concept

Create individual explanatory sections for:

1. Classes and objects

Explain the difference between a class and an object. Show an actual Java class definition; explain fields, constructors, and methods; show object creation; identify where objects are instantiated; and explain object state and behaviour. Use actual classes such as `Transaction`, `Account`, `Budget`, and `SavingsGoal`.

2. Encapsulation

Explain private fields and controlled access. Show getters, constructors, validation methods, and appropriate setters. Explain how invalid financial states are prevented.

3. Abstraction

Demonstrate interfaces and abstract classes. Explain hidden implementation details. Show actual code using interfaces such as a transaction-processing or financial-calculation abstraction and explain how abstraction reduces coupling.

4. Inheritance

Include a justified inheritance hierarchy where appropriate. For example, use an abstract `FinancialTransaction` superclass with specialised `IncomeTransaction` and `ExpenseTransaction` subclasses if compatible with the persistence architecture. Explain inherited fields/methods, show the actual `extends` relationship, and explain when inheritance is useful and when composition is preferable. Do not force meaningless inheritance.

5. Polymorphism

Demonstrate method overriding and runtime polymorphism. Show a common interface implemented by multiple transaction types. Explain dynamic method dispatch using actual Java code and identify where it is used.

6. Constructors and constructor overloading

Explain object initialisation and show actual constructors and valid overloading examples.

7. Method overloading and overriding

Explain the difference and show real examples.

8. Interfaces

Explain interface contracts and show how services and implementations use interfaces where appropriate.

9. Composition and aggregation

Explain relationships between accounts, transactions, budgets, categories, and savings goals. Show actual object relationships and database mappings.

10. Access modifiers

Explain `private`, `protected`, `public`, and package-private access and demonstrate appropriate usage.

11. Static and instance members

Explain instance fields/methods versus static members with examples.

12. `this` **and** `super`

Explain their meanings and show actual code where appropriate.

13. Exception handling

Explain custom exceptions, validation errors, appropriate `try-catch` use, and centralised API error handling.

14. Collections and generics

Explain `List`, `Set`, `Map`, and generics in the project. Show how transaction collections are processed.

15. SOLID principles

Explain Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. Connect each to a real class or design decision. Do not claim implementation unless code demonstrates it.

16. Dependency injection

Explain Spring dependency injection and how it reduces tight coupling.

17. Persistence and database integration

Explain entities, repositories, ORM, primary/foreign keys, and relationships.

18. REST APIs

Explain HTTP methods, request/response bodies, endpoints, status codes, and DTOs.

19. Unit testing

Explain test cases, assertions, edge cases, and financial calculation tests.

D. OOP demonstration playground

Include an interactive educational area where users select a concept such as encapsulation, inheritance, polymorphism, or abstraction. Display a short explanation, actual Java implementation, relevant class diagram, real-world finance example, and a demonstration of the concept's effect on behaviour.

Where practical, provide interactive examples using safe sample objects. Do not pretend Java code is executed in the browser unless a genuine Java execution environment is provided.

E. Teacher presentation mode

Include a **Presentation Mode** button. When activated, improve code readability, hide unnecessary navigation distractions, show architecture and technology stack, allow easy movement between OOP explanations and source files, provide a structured demo sequence, and include a **Return to Dashboard** button.

Suggested presentation sequence:

1. Introduce the project's purpose.
2. Explain the three-layer architecture.
3. Show `Transaction` and `Account` classes.
4. Explain classes, objects, and encapsulation.
5. Demonstrate abstraction, inheritance, and polymorphism.
6. Show services, repositories, and REST controllers.
7. Explain database schema and persistence.
8. Return to the dashboard.
9. Add a transaction live.
10. Show the updated ledger and charts.
11. Demonstrate a budget warning and savings goal.

The Specs section must contain college-level explanations tied to the actual implementation, not only textbook definitions.

18. Sample Data and Demo Mode

Prepopulate the app with a small, realistic, editable demonstration dataset clearly labelled **Demo Data**.

Use plausible example transactions such as:

- Monthly tuition income: ■9,000.
- Additional tutoring income: ■2,500.
- Occasional extra income: ■1,200.
- Groceries: ■1,500.
- Transportation: ■800.
- Educational expenses: ■1,000.
- Entertainment: ■600.
- Savings contribution: ■2,000.

These are illustrative examples, not claims about my actual finances.

Generate a coherent sample dataset covering several months so charts and month-to-month comparisons can be demonstrated. Every sample transaction must have a valid date, account, category, amount, and type. Generate opening balances and chronological transactions consistently so balances and ledger totals reconcile.

Use the same sample data across dashboard, ledger, budgets, reports, and charts. Never independently hardcode different figures on different pages.

Provide **Load Demo Data**, **Edit Demo Data**, **Clear Demo Data**, and **Reset Demo Data** controls. Clearly distinguish demo records from actual user records. Confirm before clearing or resetting. Never overwrite real user records when loading/resetting sample data.

19. Persistence and Data Safety

Financial records must persist after page refreshes and application restarts. Use the database as the authoritative source.

Implement persistent H2 file storage, database migrations or a clearly defined schema initialisation strategy, reliable CRUD operations, consistent recalculation after changes, validation, centralised exception handling, transactional database operations, CSV export, and backup/restore where practical.

Do not claim data is permanently stored if it exists only in frontend state or temporary memory.

Document that this local academic project is intended for personal use and demonstration. Do not expose the backend or financial data publicly without proper authentication and security controls.

20. Design and User Experience

Use a polished, modern finance dashboard:

- Dark navy or charcoal background with subtle green accents.
- Optional light mode.
- Readable typography.
- Rounded cards and clean layouts.
- Consistent spacing.
- Subtle animations.
- Professional icons.

- Interactive charts.
- Helpful tooltips.
- Clear status labels.
- Friendly confirmation messages.
- Responsive design for laptop, tablet, and mobile.

Keep the interface welcoming and easy to understand. Avoid excessive decoration, clutter, unreadable text, and unnecessary animations.

Include loading indicators, success notifications, validation messages, confirmation dialogs, empty/error states, keyboard-accessible controls, and responsive tables/forms.

21. Backend API Design

Create documented REST endpoints with consistent conventions. Examples:

- `GET /api/dashboard/summary`
- `GET /api/dashboard/charts`
- `GET /api/transactions`
- `POST /api/transactions`
- `GET /api/transactions/{id}`
- `PUT /api/transactions/{id}`
- `DELETE /api/transactions/{id}`
- `GET /api/ledger`
- `GET /api/accounts`
- `POST /api/accounts`
- `GET /api/budgets`
- `POST /api/budgets`
- `GET /api/savings-goals`
- `POST /api/savings-goals`
- `GET /api/reports/monthly`
- `GET /api/insights`
- `GET /api/profile`
- `PUT /api/profile`

Add update/delete endpoints where needed. Implement proper status codes, validation, DTOs, error responses, and date-range parameters. Do not expose JPA entities indiscriminately as API request/response objects.

22. Testing and Financial Correctness

Write unit tests for income/expense/net cash-flow/savings-rate/budget-utilisation calculations; account balances; transfers; transaction editing/deletion; ledger running balances; invalid amounts; zero-income/zero-budget cases; duplicate transaction prevention; date filtering; and savings goal progress.

Ensure:

- Transfers between accounts do not change total wealth.
- Savings goal allocations do not double-count assets.
- Editing/deleting a transaction recalculates dependent figures correctly.

- BigDecimal comparisons and suitable rounding rules are used in tests.

Run the test suite and fix failures before declaring the application complete.

23. Project Structure and Deliverables

Create an organised project similar to:

```
rupeewise/
 • backend/
 • src/main/java/.../
 • RupeeWiseApplication.java
 • model/
 • entity/
 • dto/
 • repository/
 • service/
 • service/impl/
 • controller/
 • exception/
 • config/
 • analytics/
 • src/main/resources/
 • application.properties
 • static/
 • src/test/java/.../
 • pom.xml
 • docs/
 • architecture.md
 • database-schema.md
 • api-documentation.md
 • oop-explanations.md
 • scripts/
 • start.bat
 • start.sh
 • README.md
```

The exact structure may be improved if a better architecture is justified.

Deliver complete Java backend, frontend source code, database configuration, sample-data initialisation, unit tests, Specs source-code catalogue, documentation, Maven configuration, Windows startup script, and README with setup/usage instructions.

The Windows startup script should verify Java availability, build the application if necessary, display useful errors, and launch it. Document the required Java version and manual start command. Make it straightforward to run locally on Windows.

24. GitHub Compatibility

Structure the project for a GitHub repository. Include `.gitignore`, README, installation/build instructions, demo screenshots or a documented screenshot-generation process, and local backend instructions.

Explain that GitHub Pages does not execute a Java Spring Boot backend. If a live hosted version is needed, explain how to deploy frontend and backend separately. Do not falsely claim that uploading a Java project to GitHub automatically makes it executable as a website.

25. Development Workflow

Follow this sequence:

1. Establish architecture and data model.
2. Implement Java domain classes, interfaces, justified inheritance, and core OOP design.
3. Implement repositories, database persistence, services, financial calculations, and tests.
4. Implement REST controllers, validation, exception handling, and tests.
5. Build frontend and connect every interactive control to the real backend.
6. Implement dashboard, charts, ledger, budgets, savings, reports, and analytics.
7. Implement Specs, actual source-code explorer, OOP explanations, and presentation mode.
8. Integrate demo data, verify financial consistency, and test workflows.
9. Prepare startup scripts, GitHub instructions, and documentation.

Do not stop after architecture, a few sample classes, or a visually appealing dashboard. If the application cannot fit into one generation, divide work into clearly defined stages and maintain consistent class names, APIs, database schemas, and file paths. Do not silently replace unfinished functionality with mock data or nonfunctional buttons.

26. Final Acceptance Criteria

The project is complete only when:

- All currency values are INR.
- The website runs locally.
- Java backend starts successfully.
- Frontend communicates with Java backend.
- Income and expenses can be created, edited, and deleted.
- Account and ledger balances reconcile.
- Transfers do not inflate income/expenses.
- Dashboard cards and charts reflect persisted data.
- Budgets and savings goals work.
- Sample data is realistic, editable, and internally consistent.
- Data persists after application restart.
- Specs displays actual project source code.
- Major OOP concepts are explained using the implementation.
- Presentation Mode works.
- Tests and documentation are included.
- No essential button is merely decorative.

- No feature is falsely presented as implemented.
- Project can be built and demonstrated on a Windows laptop.

Start by creating the complete project architecture and implement systematically. Prioritise correctness, meaningful Java OOP usage, maintainability, usability, and a polished live demonstration experience.